

```

• {
•     QApplication app(argc, argv);
•     QPushButton *button = new QPushButton("Bye");
•     QObject::connect(button, SIGNAL(clicked()),
•                       &app, SLOT(quit()));
•     button->show();
•     return app.exec();
• }

```

Let's analyse how this code is different from Hello Digit! Qt's widgets emit signals to indicate that a user action or a change of state has occurred. For instance, QPushButton emits a clicked() signal when the user clicks the button. A signal can be connected to a function (called a slot in that context) so that when the signal is emitted, the slot is automatically executed. In our example, we connect the button's clicked() signal to the QApplication object's quit() slot. The SIGNAL() and SLOT() macros are part of the syntax.

The Output

Simple Widget Layout

In this section, we will create a small example application that demonstrates how to use layouts to manage the geometry of widgets in a window and how to use signals and slots to synchronize two widgets. The application asks for the user's birth year which the user can enter by manipulating either a spin box or a slider.

Code

```

• #include <QApplication>
• #include <QHBoxLayout>
• #include <QSlider>
• #include <QSpinBox>
• int main(int argc, char *argv[])
• {
•     QApplication app(argc, argv);
•     QWidget *window = new QWidget;
•     window->setWindowTitle("Enter your birth year");
•     QSpinBox *spinBox = new QSpinBox;
•     QSlider *slider = new QSlider(Qt::Horizontal);
•     spinBox->setRange(1900, 2000);
•     slider->setRange(1900, 2000);

```